Моніторинг подій OSINT у Telegram та геоприв’язка інцидентів

Заняття №14

AI для OSINT і розвідки

Лектор
Василь Попов

ПЛАН ЗАНЯТТЯ



- Інструмент: Groupint (OSINT for Ukraine)
- Головна тема: моніторинг каналів Telegram, не лише груп
- Результат: структуровані інциденти з координатами на карті
- Формат: лекція + демо пайплайну Incidents
- Документація в репозиторії: `docs/incidents/overview.md`

ПІСЛЯ ЛЕКЦІЇ ВИ ЗМОЖЕТЕ:

- Зібрати watchlist каналів і масово імпортувати списки
- Налаштувати ключові слова (глобальні та по каналу)
- Увімкнути scheduler або робити ручний fetch
- Пройти повний пайплайн від Message до Incident (слайд 9)
- Читати карту Folium і перевіряти геолокацію вручну
- Експортувати GeoJSON / JSON / CSV для подальшого аналізу

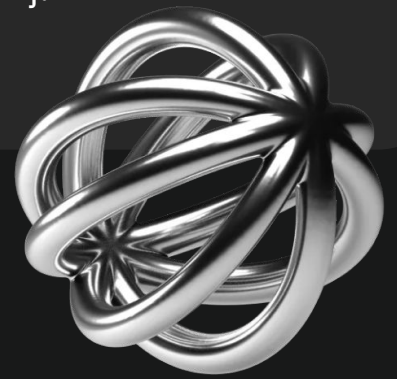
OSINT І «ПОДІЇ»

OSINT — відкриті джерела: публічні канали, пости, метадані часу.

Подія (інцидент) — звіт про ситуацію з наслідками: обстріл, евакуація, пошкодження інфраструктури.

Навіщо структура: не архів усіх постів, а картка події: що, де, коли, з якого каналу.

Вихід для аналітика: шар на карті + зв’язок з первинним повідомленням у Neo4j.



ЧОМУ TELEGRAM-КАНАЛИ

Оперативні текстові оновлення з регіонів у реальному часі

Доступ через користувацький API (Telethon), якщо ваш акаунт бачить канал

Публічні `@channel` та приватні — різна складність підключення

ВИКЛИК 1

високий обсяг шуму (реклама, репости, коментарі)

ВИКЛИК 2

неоднозначні назви населених пунктів у тексті

ВИКЛИК 3

дублювання однієї новини в кількох каналах

АНАЛІЗ СОЦІАЛЬНИХ МЕРЕЖ

Моніторинг + геолокація → ядро продукту Incidents:

Хронологія подій на території (фільтр за датою на карті)

Просторовий кластер — де сплеск повідомлень перетворюється на точки

Прослідковуваність: Message → Incident → канал (Group)

Основа для звітів, брифінгів, експорту в GIS або Atlos

Не замінює супутник чи свідчення — швидкий шар гіпотез

GROUPINT: ДВА РЕЖИМИ

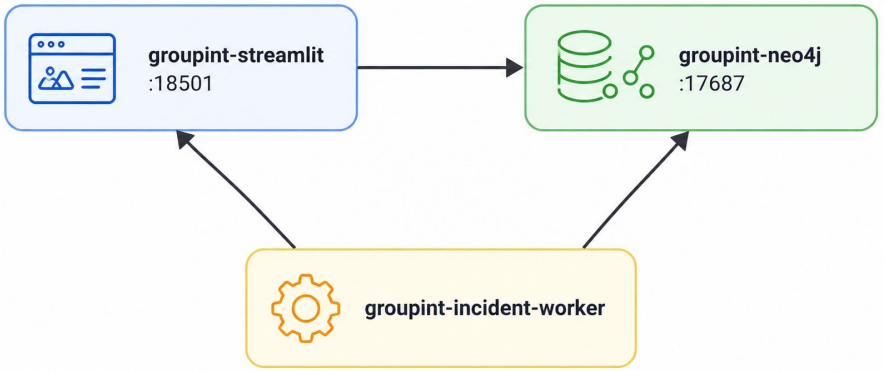
Режим	Призначення	Сьогодні
Головна сторінка	Учасники груп, MEMBER_OF, endorsements, Gephi	Огляд
Incidents	Watchlist, LLM-пайплайн, карта подій	Фокус

Обидва режими пишуть у один Neo4j, але різні задачі OSINT.

Incidents не потребує scrape учасників групи — лише пости каналів.

АРХІТЕКТУРА DOCKER (ОГЛЯД)

Три контейнери на одній мережі `groupint-net`:



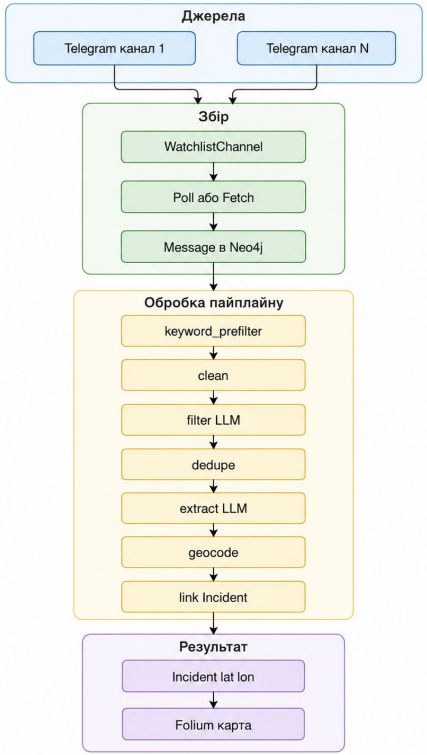
groupint-streamlit — UI Incidents, http://localhost:18501

groupint-neo4j — Bolt :17687, Browser :17474

groupint-incident-worker — scheduler + fetch + пайплайн у фоні

ПАЙПЛАЙН END-TO-END (ДІАГРАМА)

Повний ланцюг обробки одного поста (див. також слайди 22–28):



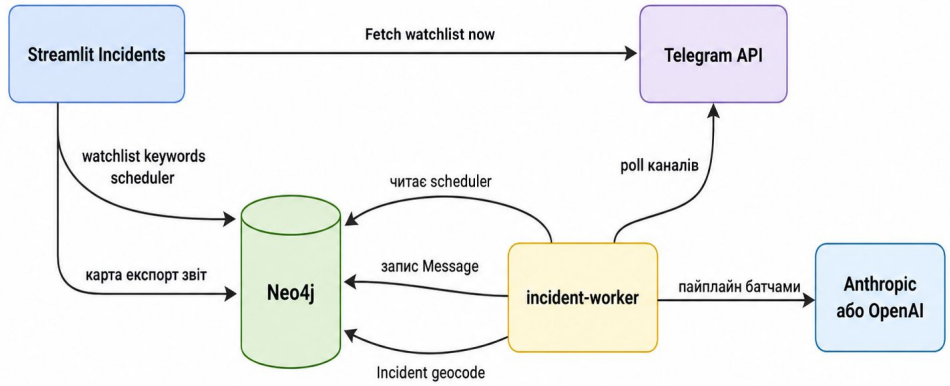
Ручні кроки в UI: Fetch watchlist now, Run pipeline now.

Автоматично: worker за scheduler — fetch + (опційно) pipeline.

Код: `core/incidents/pipeline.py` — порядок етапів фіксований.

ПАЙПЛАЙН END-TO-END (ДІАГРАМА)

Neo4j — спільна «черга станів» повідомлень; UI і worker не дублюють логіку в пам’яті.



UI — конфігурація, OTP Telegram, карта, експорт

worker — опитування за розкладом; батчі LLM без відкритого браузера

Важливо: scheduler у UI не замінює worker — потрібен запущений контейнер

OPSEC

- Окремий Telegram-акаунт для OSINT-задач (за можливості)
- api_id / api_hash — тільки `.env` або `.streamlit/secrets.toml`, не в git
- StringSession (файл `.string`) — як пароль сесії; не надсилати в чати
- Ротація ключів LLM при витоку
- Docker volume для сесій — бекап лише на захищеному носії
- Не демонструйте реальні ключі в залі під час лекції

ПЕРЕДУМОВИ

Компонент	Навіщо
Docker	Neo4j + Streamlit + worker одною командою
api_id + api_hash	Telethon на my.telegram.org
LLM API key	filter + extract (Anthropic або OpenAI)
GOOGLE_MAPS_API_KEY	опційно; інакше Nominatim

Без LLM: fetch працює, інциденти на карті не з’являться.

ЗАПУСК СТЕКУ

```
cp .env.example .env
# TELEGRAM_*, ANTHROPIC_API_KEY або OPENAI_API_KEY
./scripts/up-desktop.sh
```

Groupint: http://localhost:18501

01

Neo4j Browser: http://localhost:17474 (`NEO4J_AUTH=none`)

02

Перевірка: `curl -sf http://localhost:18501/_stcore/health`

03

Логи worker: `docker logs -f groupint-incident-worker`

04

АВТОРИЗАЦІЯ TELEGRAM

Вхід: api_id, api_hash, phone (secrets або форма).

Кроки на Incidents:

01

Create / connect Telegram client

02

Код OTP у додатку Telegram

03

Verify secret code → StringSession на диск

Вихід: сесія в `GROUPINT_SESSIONS_DIR`; auto-reconnect після F5.

Помилка: `SessionInUseError` — одна активна сесія на номер.

WATCHLIST КАНАЛІВ

WatchlistChannel у Neo4j — ваш перелік джерел.

- Поле channel_ref: `@name`, `https://t.me/...`
- Enabled — чи входить канал у poll
- last_polled_at — коли востаннє читали пости
- Один канал можна тимчасово вимкнути без видалення
- Зв’язок з Group після першого успішного fetch

МАСОВИЙ ІМПОРТ КАНАЛІВ

Expander Add multiple channels (paste or upload):



Навіщо: старт моніторингу 20–50 каналів за хвилини, не вручну.

01

Текст: один канал на рядок

02

.txt / .csv / .json — парсер `watchlist_channels.py`

03

Add all to watchlist — merge у Neo4j

04

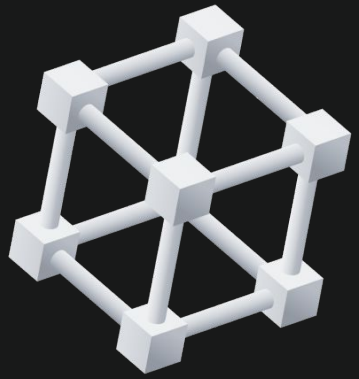
Дублікати обробляються DAL (оновлення, не «падіння»

ГЛОБАЛЬНІ КЛЮЧОВІ СЛОВА

- IncidentMonitorConfig — singleton у Neo4j.
- Список термінів (кома або новий рядок)
- Логіка OR: достатньо одного збігу в тексті поста
- Enable global keywords — увімкнути/вимкнути без видалення списку
- Save global keywords — зберегти в базу
- Порожній список + вимкнено → глобальний фільтр не діє

КЛЮЧОВІ СЛОВА ПО КАНАЛУ

У expander кожного каналу:



01

Локальний список (своя лексика джерела)

02

Also use global keywords — об’єднання списків

03

use_global = false — тільки локальні слова

04

Обидва вимкнені → усі повідомлення йдуть у пайплайн (слайд 9)

Приклад: глобально «обстріл»; для морського каналу — «корабель», «порт».

ПЛАНУВАЛЬНИК ОПИТУВАННЯ

Блок Message fetch scheduler (зберігається в Neo4j):

- ✔ scheduler_enabled — worker опитує автоматично
- ✔ fetch_interval_sec — мінімум секунд між циклами (напр. 1800)
- ✔ run_pipeline_after_fetch — LLM одразу після ingest
- ✔ last_fetch_at — час останнього poll
- ✔ Save scheduler — застосувати для worker
- 🐞 Рекомендація: спочатку ручний fetch, потім scheduler.

РУЧНЕ ОПИТУВАННЯ (INGEST)

Кнопка Fetch watchlist now:

Використовує Telethon-сесію з браузера (UI)

Читає нові пости з усіх enabled каналів

Записує Message у Neo4j (блок ingest на слайді 9)

Оновлює last_fetch_at

Не запускає LLM, якщо не натиснуто Run pipeline / немає run_pipeline_after_ fetch

Демо: перший fetch після додавання каналу.

ФОНОВИЙ INCIDENT-WORKER

Контейнер `groupint-incident-worker`, скрипт `incident-worker.py`:

Кожен цикл: прочитати scheduler → чи минув interval?

poll_all_watchlist через Telegram (env `TELEGRAM_*`)

Опційно `run_pending_pipeline` після fetch

Без LLM key — ingest тихий, без спаму помилок у логах

Діагностика: `docker logs -f groupint-incident-worker`

INGEST: MESSAGE У NEO4J

Вхід: нові пости з Telegram API.

Вихід: вузол `Message` + зв’язок з `Group` (канал).

Поля: `text`, `date`, `message_id`, `group_id`, `incident_pipeline_stage`.

Інкрементальність: існуючі `message_id` не перезаписуються з нуля.

Стадія після fetch: готово до `keyword_prefilter` (слайд 9).

ЕТАП 1: KEYWORD_PREFILTER

Вхід: Message з сирим текстом, ще без `text_clean`.

Дія: `effective_keywords()` — global + per-channel (слайди 16–17).

Вихід: пропуск далі або відсікання без LLM.

Навіщо: економія токенів; фокус на релевантних постах.

Код: `core/incidents/keyword_prefilter.py`. Див. слайд 9.

ЕТАП 2: CLEAN

Вхід: повідомлення, що пройшли prefilter.

Дія: нормалізація тексту; за наявності LLM — `CLEANER_SYSTEM` prompt.

Вихід: поле `text_clean`, stage `cleaned`.

Без LLM: копія сирого тексту.

Навіщо: стабільний вхід для filter/extract.

ЕТАП 3: FILTER (INCIDENT_FILTER)

Вхід: `text_clean`.

Дія: LLM JSON — чи mappable (звіт про інцидент)?

Вихід: `incident_checked=1` або `-1`, stage `filtered`.

Не інцидент — пайплайн зупиняється для цього Message.

Провайдер: `INCIDENT_LLM_PROVIDER` + відповідний API key.

ЕТАП 4: DEDUPE

Вхід: повідомлення, позначені як інцидентні.

Дія: `run_dedupe_batch` — кластеризація схожих подій.

Вихід: менше дублікатів на карті при репостах.

Навіщо: один вибух у п’яти каналах ≠ п’ять незалежних точок без dedupe.

Порядок у коді: dedupe перед extract (`pipeline.py`).

ЕТАП 5: EXTRACT

Вхід: `text_clean` після filter + dedupe.

Дія: LLM `EXTRACT_SYSTEM` → JSON.

Вихід на Message: `category`, `location_text`, stage `extracted`.

Приклад location_text: «Костянтинівка, Донецька обл.»

Навіщо: структуровані поля перед geocode.

ЕТАП 6: GEOCODE

Вхід: `location_text` + контекст `text_clean`.

Дія: Google Maps API або Nominatim.

Вихід: `lat`, `lon` на Message; stage `geocoded` або `geocode_failed`.

Критично для карти — без координат Incident не має сенсу на Folium.

Див. слайд 9 — блок geocode → link.

ЕТАП 7: LINK → INCIDENT

Вхід: Message з `lat`, `lon`, category, summary.

Дія: `run_link_incidents_batch` → `merge_incident_from_message`.

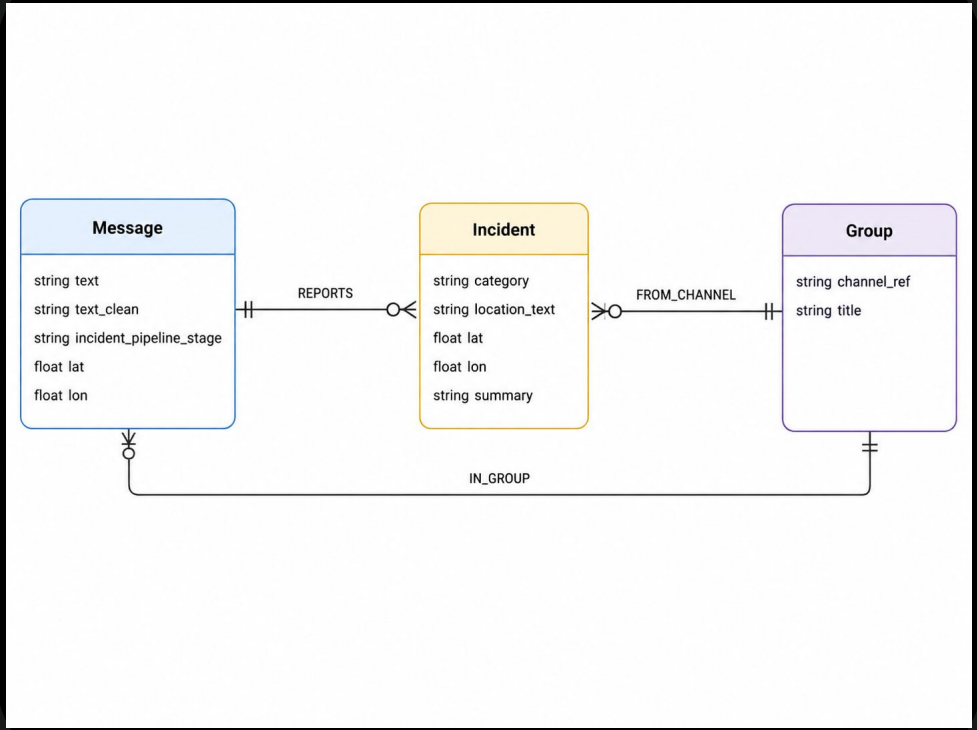
Вихід: вузол Incident + `(Message)-[:REPORTS]->(Incident)`.

Також: `(Incident)-[:FROM_CHANNEL]->(Group)`.

Після цього — точка з’являється на карті (слайд 31).

МОДЕЛЬ ДАНИХ NEO4J (ДІАГРАМА)

Зв’язки між сутностями після link:



Вузол	Ключові поля
Message	text_clean, lat, lon, pipeline_stage
Incident	category, summary, occurred_at, atlos_slug
Group	channel_ref, title

КАРТА ІНЦИДЕНТІВ (FOLIUM)

Вкладка Incidents — Incident map:

- Маркери з `Incident.lat` / `lon`
- Колір / фільтр за категорією
- Фільтр за діапазоном дат
- Popup: summary, location_text, посилання на джерело
- Оновлення після Run pipeline / worker
- Не GIS-рівня — огляд і брифінг; для QGIS — експорт GeoJSON.

ВЕРИФІКАЦІЯ ГЕОЛОКАЦІЇ

Обов’язковий крок аналітика:



01

Порівняти `location_text` з текстом поста

02

Перевірити координати на OSM / супутнику

03

Однойменні села — типова помилка geocoder

04

LLM може видумати район або переплутати область

Карта = гіпотеза для подальшого підтвердження.

Документуйте рівень впевненості в звіті команди.

ЕКСПОРТ ДАНИХ

З UI (з урахуванням фільтрів карти):

01	02	03
GEOJSON — QGIS, ARCGIS	JSON — PYTHON / JQ PIPELINES	CSV — EXCEL, СТАТИСТИКА

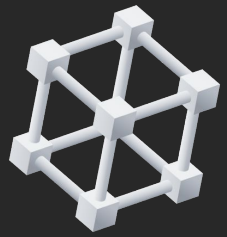
Навіщо: архів епізоду моніторингу, передача в інші команди.

Експорт не замінює Neo4j для історичного граф-аналізу.

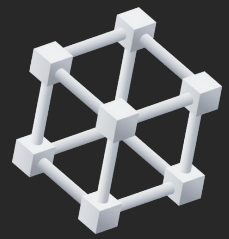
ЕКСПОРТ У ATLOS (ОПЦІЙНО)

[Atlos](https://atlos.org) — спільна платформа інцидентів.

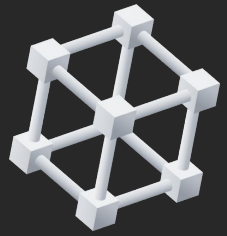
API v2: `POST /api/v2/incidents/new`



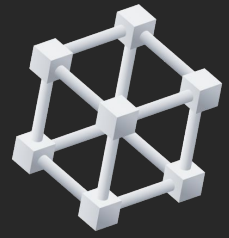
UI: Export to Atlos, Test connection



Локально: `./scripts/up-full.sh`, http://localhost:13000



`atlos_slug` на Incident — не дублювати експорт



Опційний слайд — якщо організація не використовує Atlos, пропустіть.

ЗВІТ РОЗВІДКИ

Generate intelligence report — текстовий підсумок:

За тими ж фільтрами, що й карта

Категорії, локації, хронологія словами

Для брифінгу керівництву / партнерам

Перевіряйте факти по первинних Message

LLM у звіті може узагальнити з помилками



ЧЕКЛИСТ ДЕМО (УЗГОДЖЕНО З ПАЙПЛАЙНОМ)

- ✓ `up-desktop.sh` → 2. OTP Incidents → 3. Watchlist (+ bulk?)
- ✓ Global keywords → 5. Per-channel (опційно)
- ✓ Save scheduler або Fetch now
- ✓ Run pipeline now (слайд 9)
- ✓ Карта + клік маркера → 9. Верифікація 1–2 точок
- ✓ Експорт JSON / GeoJSON

Орієнтир: 15–20 хв живого демо.

ТИПОВІ ПОМИЛКИ

Симптом	Що перевірити
Немає точок на карті	LLM key, Run pipeline, geocode_failed
Fetch 0 messages	Канал enabled? Акаунт бачить канал?
Scheduler не працює	worker running? scheduler_enabled?
Порожня карта після фільтра	Діапазон дат, категорія
Worker auth error	TELEGRAM_* в `.env` для worker

ОБМЕЖЕННЯ СИСТЕМИ



Якість подій = якість тексту в каналі



LLM: false positive / false negative



Geocode: омоніми, відсутність адреси в пості



Telegram rate limits — великий watchlist + короткий interval



Вартість токенів LLM при вимкненому keyword filter



Не детектує маніпуляції зображеннями (тільки текст поста)

РУЧНИЙ VS АВТОМАТИЧНИЙ OSINT

Ручний	Groupint Incidents
Скрол стрічки каналів	Scheduler + Neo4j
Таблиця в Excel	Incident + Message link
Точки на карті вручну	Folium + geocode pipeline

Автоматизація масштабує охоплення; аналітик залишається на верифікації (слайд 31).

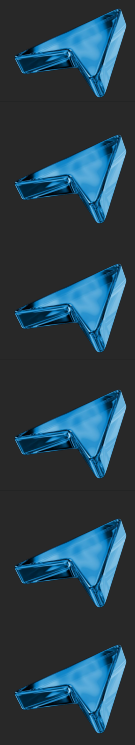
НАВЧАЛЬНИЙ КЕЙС

Сценарій: 3 канали, global: `обстріл`, `удари`, interval 30 хв.

Bulk import watchlist	Увімкнути run_pipeline_after_fetch	Дочекатися 2 циклів worker (або 2× Fetch + Run pipeline)
Фільтр карти за категорією	Верифікувати 2 маркери на супутниковій карті	У звіті: одна підтверджена, одна відхилена — чому

РЕСУРСИ

- `docs/index.md` — хаб документації
- `docs/incidents/overview.md` — пайплайн
- `docs/incidents/keywords-and-scheduler.md`
- `docs/tutorial-full-workflow-uk.md` — повний ланцюг UK
- GitHub: github.com/OSINT-for-Ukraine/groupint
- Issues — технічна підтримка



ПРАКТИКА ТА ПИТАННЯ

Завдання (45–60 хв): 1 канал, 5 keywords, Fetch + Run pipeline, JSON з 3 інцидентами, абзац верифікації гео.



Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ